

Chapter 19 — Backtracking

0. What You Will Learn in This Chapter

Backtracking is one of the most important patterns for generating possibilities and solving constraint-based problems.

It is built on recursion.

The core idea is:

choose
explore
unchoose

Backtracking tries a choice, explores where it leads, then undoes the choice and tries another option.

By the end of this chapter, you should understand:

- What backtracking is
- Why backtracking exists
- How it differs from normal recursion
- How to write choose / explore / unchoose code
- How to generate subsets
- How to generate combinations
- How to generate permutations
- How to handle duplicates
- How to solve combination sum problems
- How to solve grid search problems
- How to solve partitioning problems
- How to solve N-Queens
- How to prune invalid branches
- How to explain exponential complexity
- How to recognize backtracking in interviews

Core problems covered in this chapter:

1. Subsets
 2. Subsets II
 3. Permutations
 4. Permutations II
 5. Combinations
 6. Combination Sum
 7. Combination Sum II
 8. Letter Combinations of a Phone Number
 9. Generate Parentheses
 10. Palindrome Partitioning
 11. Word Search
 12. N-Queens
 13. Sudoku Solver
 14. Restore IP Addresses
 15. Beautiful Arrangement
 16. Matchsticks to Square
 17. Partition to K Equal Sum Subsets
 18. Expression Add Operators
-

1. What Is Backtracking?

Backtracking is a recursive search technique.

It builds a partial solution step by step.

At each step:

19. choose an option
20. recursively explore
21. undo the choice

Example:

Generate subsets of:

[1, 2]

Choices:

include 1 or exclude 1
include 2 or exclude 2

Result:

[]
[1]
[2]
[1,2]

Backtracking explores all valid choices.

2. Why Backtracking Exists

Some problems require exploring many possible configurations.

Examples:

- all subsets
- all permutations
- all combinations
- all valid parentheses strings
- all paths in a grid
- all board placements
- all partitions
- all ways to reach target

There is usually no simple one-pass solution because the answer itself may contain exponentially many results.

Backtracking provides a structured way to explore this search space.

3. Core Need / Why Use Backtracking Over Other Patterns?

3.1 Why Not Basic Recursion Only?

Backtracking is recursion with state changes that must be undone.

Normal recursion:

```
return solve(n - 1);
```

Backtracking:

```
path.push_back(choice);  
dfs(...);  
path.pop_back();
```

The pop_back() is the backtracking step.

3.2 Why Not Dynamic Programming?

DP is for counting or optimizing when subproblems repeat.

Backtracking is for generating actual solutions or exploring constraints.

Example:

- Count number of ways: often DP
- Generate all ways: often backtracking

3.3 Why Not Greedy?

Greedy makes one best local choice.

Backtracking tries multiple choices because local choice may not lead to valid global solution.

Example:

N-Queens cannot be solved by simply choosing first available column greedily.

3.4 Why Not BFS?

BFS can explore states too, but backtracking is usually simpler when constructing solutions depth-first and undoing choices.

4. When to Use Backtracking

Use backtracking when the problem asks for:

4.1 Generate All Possibilities

Clues:

- all subsets
- all permutations
- all combinations
- all valid strings

Examples:

- Subsets
- Permutations
- Combinations
- Generate Parentheses

4.2 Constraint Satisfaction

Clues:

- place items subject to rules
- solve board
- fill grid
- no conflicts

Examples:

- N-Queens
- Sudoku Solver

4.3 Search With Path

Clues:

- find word in grid
- path cannot reuse cell
- choose next move

Examples:

- Word Search
- Rat in a Maze style problems

4.4 Partitioning

Clues:

- split string into valid parts
- partition into palindromes
- divide into k equal groups

Examples:

- Palindrome Partitioning
- Restore IP Addresses
- Partition to K Equal Sum Subsets

4.5 Target Sum Combinations

Clues:

- choose numbers to reach target
- combination sum
- repeated use allowed or not allowed

Examples:

- Combination Sum
 - Combination Sum II
-

5. When NOT to Use Backtracking

5.1 Need Only One Simple Linear Answer

Use traversal, hashing, two pointers, etc.

5.2 Need Count With Large Constraints

If constraints are large and only count is needed, use DP or combinatorics.

5.3 Need Shortest Path

Use BFS if all moves have equal cost.

5.4 Choices Are Locally Deterministic

If there is a clear greedy proof, use greedy.

5.5 Input Size Too Large for Exponential Search

Backtracking is usually exponential.

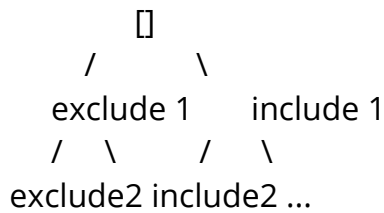
If n is large, look for DP, greedy, math, or graph algorithms.

6. Core Intuition

Backtracking explores a decision tree.

At each node in the tree, you make a choice.

Example for subsets of [1,2,3]:



Each root-to-leaf path represents one possible solution.
Backtracking is DFS over this decision tree.

7. Mental Models

7.1 Choose / Explore / Unchoose

This is the most important model.

```

path.push_back(choice); // choose
dfs(nextState);         // explore
path.pop_back();        // unchoose
  
```

7.2 Decision Tree

Each recursive call is a node in the tree.
Each choice creates a branch.

7.3 Path Represents Current Partial Solution

path stores the choices made so far.
When path becomes complete or valid, copy it into result.

7.4 Pruning

Stop exploring branches that cannot lead to valid answers.
Example:
If current sum exceeds target, return.

8. Pattern Recognition Checklist

Ask:

22. Does the problem ask to generate all solutions?
23. Are there multiple choices at each step?
24. Do I need to undo choices?
25. Is there a partial solution/path?
26. Are there constraints to check?
27. Can invalid branches be pruned?
28. Is input size small enough for exponential search?
29. Does order matter?
30. Are duplicates present?
31. Can the same element be reused?

If yes, backtracking is likely useful.

9. C++ Tools Needed

9.1 Path Vector

```
vector<int> path;
```

9.2 Result Vector

```
vector<vector<int>> result;
```

9.3 Push and Pop

```
path.push_back(x);  
dfs(...);  
path.pop_back();
```

9.4 Visited Array

Used for permutations and grid paths.

```
vector<bool> used(n, false);
```

9.5 Sorting for Duplicate Handling

```
sort(nums.begin(), nums.end());
```

Then skip duplicates carefully.

9.6 Passing by Reference

Avoid copying:

```
void dfs(vector<int>& nums, vector<int>& path, vector<vector<int>>& result)
```

10. Core Templates

Template 1 — Subsets, Include/Exclude

```
void dfs(vector<int>& nums, int index, vector<int>& path, vector<vector<int>>& result) {  
    if (index == nums.size()) {  
        result.push_back(path);  
        return;  
    }  
  
    dfs(nums, index + 1, path, result);  
  
    path.push_back(nums[index]);  
    dfs(nums, index + 1, path, result);  
    path.pop_back();  
}
```

Template 2 — Subsets, For-Loop Style

```
void dfs(vector<int>& nums, int start, vector<int>& path, vector<vector<int>>& result) {
```

```

    result.push_back(path);

    for (int i = start; i < nums.size(); i++) {
        path.push_back(nums[i]);
        dfs(nums, i + 1, path, result);
        path.pop_back();
    }
}

```

Template 3 — Combinations

```

void dfs(int start, int n, int k, vector<int>& path, vector<vector<int>>& result) {
    if (path.size() == k) {
        result.push_back(path);
        return;
    }

    for (int i = start; i <= n; i++) {
        path.push_back(i);
        dfs(i + 1, n, k, path, result);
        path.pop_back();
    }
}

```

Template 4 — Permutations

```

void dfs(vector<int>& nums, vector<bool>& used, vector<int>& path,
vector<vector<int>>& result) {
    if (path.size() == nums.size()) {
        result.push_back(path);
        return;
    }

    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) {
            continue;
        }

        used[i] = true;
        path.push_back(nums[i]);

        dfs(nums, used, path, result);
    }
}

```

```
    path.pop_back();
    used[i] = false;
}
}
```

Template 5 — Combination Sum, Reuse Allowed

```
void dfs(vector<int>& candidates, int start, int target,
        vector<int>& path, vector<vector<int>>& result) {
    if (target == 0) {
        result.push_back(path);
        return;
    }

    if (target < 0) {
        return;
    }

    for (int i = start; i < candidates.size(); i++) {
        path.push_back(candidates[i]);

        dfs(candidates, i, target - candidates[i], path, result);

        path.pop_back();
    }
}
```

Reuse allowed because recursive call uses i.

Template 6 — Combination Sum, Reuse Not Allowed

```
void dfs(vector<int>& candidates, int start, int target,
        vector<int>& path, vector<vector<int>>& result) {
    if (target == 0) {
        result.push_back(path);
        return;
    }

    for (int i = start; i < candidates.size(); i++) {
        if (i > start && candidates[i] == candidates[i - 1]) {
            continue;
        }

        if (candidates[i] > target) {
```

```

        break;
    }

    path.push_back(candidates[i]);

    dfs(candidates, i + 1, target - candidates[i], path, result);

    path.pop_back();
}
}

```

Reuse not allowed because recursive call uses $i + 1$.

Template 7 — Grid Backtracking

```

bool dfs(vector<vector<char>>& board, string& word, int r, int c, int index) {
    if (index == word.size()) {
        return true;
    }

    if (r < 0 || c < 0 || r >= board.size() || c >= board[0].size()) {
        return false;
    }

    if (board[r][c] != word[index]) {
        return false;
    }

    char saved = board[r][c];
    board[r][c] = '#';

    bool found =
        dfs(board, word, r + 1, c, index + 1) ||
        dfs(board, word, r - 1, c, index + 1) ||
        dfs(board, word, r, c + 1, index + 1) ||
        dfs(board, word, r, c - 1, index + 1);

    board[r][c] = saved;

    return found;
}

```

11. Major Variations of Backtracking

Subsets	choose / not choose	Subsets
Combinations	choose next increasing item	Combinations
Permutations	choose unused item	Permutations
Combination sum	choose numbers to target	Combination Sum
String partition	choose cut position	Palindrome Partitioning
Parentheses	add valid (or)	Generate Parentheses
Grid search	move 4 directions	Word Search
Board placement	place valid piece	N-Queens
Constraint fill	fill empty cells	Sudoku Solver
K buckets	place item into group	Partition to K Equal Sum

12. Full Problem Walkthrough 1 — LeetCode 78: Subsets

12.1 Problem Summary

Given unique integers `nums`, return all possible subsets.

Example:

```
nums = [1,2,3]
```

Output:

```
[
  [],
  [1],
  [2],
  [3],
  [1,2],
  [1,3],
  [2,3],
  [1,2,3]
]
```

12.2 How to Identify This Pattern

Problem asks for all subsets.
Each element can be either:

- included
- excluded

This is backtracking.

12.3 C++ Code, For-Loop Style

```
class Solution {
public:
    void dfs(vector<int>& nums, int start,
            vector<int>& path,
            vector<vector<int>>& result) {
        result.push_back(path);

        for (int i = start; i < nums.size(); i++) {
            path.push_back(nums[i]);

            dfs(nums, i + 1, path, result);

            path.pop_back();
        }
    }

    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> result;
        vector<int> path;

        dfs(nums, 0, path, result);

        return result;
    }
};
```

12.4 Dry Run

For [1,2]:

```
[]
[1]
[1,2]
[2]
```

The function records current path at every recursion level.

12.5 Complexity

There are 2^n subsets.
Each subset can take $O(n)$ to copy.
Time:

$O(n * 2^n)$

Space:

$O(n)$

excluding output.

12.6 Common Mistakes

- Only recording at leaves
- Forgetting `path.pop_back()`
- Passing path by value unnecessarily

13. Full Problem Walkthrough 2 — LeetCode 90: Subsets II

13.1 Problem Summary

Given integer array that may contain duplicates, return all unique subsets.
Example:

`nums = [1,2,2]`

Output:

`[[], [1], [1,2], [1,2,2], [2], [2,2]]`

13.2 How to Identify This Pattern

It is subsets, but duplicates exist.
Need sort and skip duplicate choices at same recursion level.

13.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& nums, int start,
            vector<int>& path,
            vector<vector<int>>& result) {
        result.push_back(path);

        for (int i = start; i < nums.size(); i++) {
            if (i > start && nums[i] == nums[i - 1]) {
                continue;
            }
        }
    }
};
```

```

        path.push_back(nums[i]);

        dfs(nums, i + 1, path, result);

        path.pop_back();
    }
}

vector<vector<int>> subsetsWithDup(vector<int>& nums) {
    sort(nums.begin(), nums.end());

    vector<vector<int>> result;
    vector<int> path;

    dfs(nums, 0, path, result);

    return result;
}
};

```

13.4 Why i > start?

Skip duplicates only at the same decision level.

This prevents duplicate subsets, but still allows choosing multiple copies across deeper levels.

13.5 Complexity

Time:

$O(n * 2^n)$

Space:

$O(n)$

excluding output.

14. Full Problem Walkthrough 3 — LeetCode 46: Permutations

14.1 Problem Summary

Given unique integers, return all permutations.

Example:

nums = [1,2,3]

Output includes:

[1,2,3], [1,3,2], [2,1,3], ...

14.2 How to Identify This Pattern

Need all orderings.

At each position, choose one unused number.

Use backtracking with used.

14.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& nums,
            vector<bool>& used,
            vector<int>& path,
            vector<vector<int>>& result) {
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }

        for (int i = 0; i < nums.size(); i++) {
            if (used[i]) {
                continue;
            }

            used[i] = true;
            path.push_back(nums[i]);

            dfs(nums, used, path, result);

            path.pop_back();
            used[i] = false;
        }
    }

    vector<vector<int>> permute(vector<int>& nums) {
        vector<vector<int>> result;
        vector<int> path;
        vector<bool> used(nums.size(), false);

        dfs(nums, used, path, result);
    }
};
```

```
        return result;
    }
};
```

14.4 Complexity

There are $n!$ permutations.

Each copy takes $O(n)$.

Time:

$O(n * n!)$

Space:

$O(n)$

excluding output.

14.5 Common Mistakes

- Forgetting to reset `used[i]`
- Forgetting to pop path
- Using start like combinations; permutations need all unused positions

15. Full Problem Walkthrough 4 — LeetCode 47: Permutations II

15.1 Problem Summary

Given numbers that may contain duplicates, return unique permutations.

Example:

`nums = [1,1,2]`

Output:

`[1,1,2], [1,2,1], [2,1,1]`

15.2 How to Identify This Pattern

Permutation with duplicates.

Sort first.

Skip duplicate number if previous same number has not been used in this branch.

15.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& nums,
            vector<bool>& used,
            vector<int>& path,
```

```

        vector<vector<int>>& result) {
    if (path.size() == nums.size()) {
        result.push_back(path);
        return;
    }

    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) {
            continue;
        }

        if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) {
            continue;
        }

        used[i] = true;
        path.push_back(nums[i]);

        dfs(nums, used, path, result);

        path.pop_back();
        used[i] = false;
    }
}

vector<vector<int>> permuteUnique(vector<int>& nums) {
    sort(nums.begin(), nums.end());

    vector<vector<int>> result;
    vector<int> path;
    vector<bool> used(nums.size(), false);

    dfs(nums, used, path, result);

    return result;
}
};

```

15.4 Duplicate Skip Explanation

Condition:

`i > 0 && nums[i] == nums[i - 1] && !used[i - 1]`

means:

Do not choose the second duplicate before choosing the first duplicate at the same level.

This prevents duplicate permutations.

15.5 Complexity

Worst-case:

$O(n * n!)$

With duplicates, output count is smaller.

16. Full Problem Walkthrough 5 — LeetCode 77: Combinations

16.1 Problem Summary

Given n and k , return all combinations of k numbers from 1 to n .

Example:

$n = 4$

$k = 2$

Output:

[1,2], [1,3], [1,4], [2,3], [2,4], [3,4]

16.2 How to Identify This Pattern

Need choose k numbers.

Order does not matter.

Use start index to avoid duplicates and reversed orders.

16.3 C++ Code

```
class Solution {
public:
    void dfs(int start, int n, int k,
             vector<int>& path,
             vector<vector<int>>& result) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }

        for (int i = start; i <= n; i++) {
            path.push_back(i);

            dfs(i + 1, n, k, path, result);
        }
    }
};
```

```

        path.pop_back();
    }
}

vector<vector<int>> combine(int n, int k) {
    vector<vector<int>> result;
    vector<int> path;

    dfs(1, n, k, path, result);

    return result;
}
};

```

16.4 Pruned Version

If remaining numbers are not enough, stop early.

```

for (int i = start; i <= n - (k - path.size()) + 1; i++) {
    path.push_back(i);
    dfs(i + 1, n, k, path, result);
    path.pop_back();
}

```

16.5 Complexity

Number of combinations:

$C(n, k)$

Time:

$O(k * C(n, k))$

Space:

$O(k)$

excluding output.

17. Full Problem Walkthrough 6 — LeetCode 39: Combination Sum

17.1 Problem Summary

Given distinct candidates and target.

Return all combinations where chosen numbers sum to target.

Each number can be used unlimited times.

Example:

```
candidates = [2,3,6,7]
target = 7
```

Output:

```
[2,2,3], [7]
```

17.2 How to Identify This Pattern

Need all combinations reaching target.

Choices can repeat.

Use backtracking with start.

Reuse allowed means recursive call uses same i.

17.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& candidates, int start, int target,
            vector<int>& path,
            vector<vector<int>>& result) {
        if (target == 0) {
            result.push_back(path);
            return;
        }

        if (target < 0) {
            return;
        }

        for (int i = start; i < candidates.size(); i++) {
            path.push_back(candidates[i]);

            dfs(candidates, i, target - candidates[i], path, result);

            path.pop_back();
        }
    }

    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        vector<vector<int>> result;
        vector<int> path;

        dfs(candidates, 0, target, path, result);
    }
};
```

```
        return result;
    }
};
```

17.4 Optimized With Sorting

```
sort(candidates.begin(), candidates.end());

if (candidates[i] > target) {
    break;
}
```

This prunes impossible branches.

17.5 Complexity

Exponential.

A rough upper bound depends on target and smallest candidate.

Space:

$O(\text{target} / \text{minCandidate})$

recursion depth.

17.6 Common Mistakes

- Using $i + 1$ when reuse is allowed
- Not reducing target
- Not popping path
- Generating duplicate orderings like $[2,3,2]$

18. Full Problem Walkthrough 7 — LeetCode 40: Combination Sum II

18.1 Problem Summary

Candidates may contain duplicates.

Each number can be used once.

Return unique combinations summing to target.

Example:

`candidates = [10,1,2,7,6,1,5]`

`target = 8`

Output:

`[1,1,6], [1,2,5], [1,7], [2,6]`

18.2 How to Identify This Pattern

Combination sum, but:

- duplicates exist
- each element used once

Sort and skip duplicate choices at same level.

18.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& candidates, int start, int target,
            vector<int>& path,
            vector<vector<int>>& result) {
        if (target == 0) {
            result.push_back(path);
            return;
        }

        for (int i = start; i < candidates.size(); i++) {
            if (i > start && candidates[i] == candidates[i - 1]) {
                continue;
            }

            if (candidates[i] > target) {
                break;
            }

            path.push_back(candidates[i]);

            dfs(candidates, i + 1, target - candidates[i], path, result);

            path.pop_back();
        }
    }

    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end());

        vector<vector<int>> result;
        vector<int> path;

        dfs(candidates, 0, target, path, result);

        return result;
    }
};
```

```
}  
};
```

18.4 Key Differences From Combination Sum I

duplicates in input	no	yes
reuse allowed	yes	no
recursive call	dfs(i, ...)	dfs(i + 1, ...)
duplicate skipping	not needed	needed

19. Full Problem Walkthrough 8 — LeetCode 17: Letter Combinations of a Phone Number

19.1 Problem Summary

Given digits from 2 to 9, return all possible letter combinations.

Example:

```
digits = "23"
```

Output:

```
["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

19.2 How to Identify This Pattern

Each digit has multiple choices.

Need generate all combinations.

Use backtracking.

19.3 C++ Code

```
class Solution {  
public:  
    vector<string> mapping = {  
        "", "", "abc", "def", "ghi", "jkl",  
        "mno", "pqrs", "tuv", "wxyz"  
    };  
  
    void dfs(string& digits, int index,  
            string& path,
```

```

        vector<string>& result) {
    if (index == digits.size()) {
        result.push_back(path);
        return;
    }

    string letters = mapping[digits[index] - '0'];

    for (char c : letters) {
        path.push_back(c);

        dfs(digits, index + 1, path, result);

        path.pop_back();
    }
}

vector<string> letterCombinations(string digits) {
    vector<string> result;

    if (digits.empty()) {
        return result;
    }

    string path;

    dfs(digits, 0, path, result);

    return result;
}
};

```

19.4 Complexity

If each digit has up to 4 letters and length is n:

Time:

$O(4^n * n)$

Space:

$O(n)$

excluding output.

20. Full Problem Walkthrough 9 —

LeetCode 22: Generate Parentheses

20.1 Problem Summary

Generate all valid combinations of n pairs of parentheses.

Example:

$n = 3$

Output:

`["((()))","(()())","(())()","()(())","()()()"]`

20.2 How to Identify This Pattern

Need generate all valid strings under constraints.

At each step:

- add (if open count < n
- add) if close count < open count

20.3 C++ Code

```
class Solution {
public:
    void dfs(int n, int open, int close,
             string& path,
             vector<string>& result) {
        if (path.size() == 2 * n) {
            result.push_back(path);
            return;
        }

        if (open < n) {
            path.push_back('(');
            dfs(n, open + 1, close, path, result);
            path.pop_back();
        }

        if (close < open) {
            path.push_back(')');
            dfs(n, open, close + 1, path, result);
            path.pop_back();
        }
    }
};
```

```

    }

    vector<string> generateParenthesis(int n) {
        vector<string> result;
        string path;

        dfs(n, 0, 0, path, result);

        return result;
    }
};

```

20.4 Complexity

Number of valid strings is Catalan number.

Time proportional to output size.

Space:

$O(n)$

recursion depth and path.

20.5 Common Mistakes

- Adding) when close == open
- Not limiting open $\leq n$
- Building invalid strings then filtering later

21. Full Problem Walkthrough 10 — LeetCode 131: Palindrome Partitioning

21.1 Problem Summary

Partition string so every substring is palindrome.

Return all possible palindrome partitions.

Example:

`s = "aab"`

Output:

`["a","a","b"], ["aa","b"]`

21.2 How to Identify This Pattern

Need all ways to split string.

At each index, choose next cut.

Only explore cut if substring is palindrome.

21.3 C++ Code

```
class Solution {
public:
    bool isPalindrome(string& s, int left, int right) {
        while (left < right) {
            if (s[left] != s[right]) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }

    void dfs(string& s, int start,
            vector<string>& path,
            vector<vector<string>>& result) {
        if (start == s.size()) {
            result.push_back(path);
            return;
        }

        for (int end = start; end < s.size(); end++) {
            if (!isPalindrome(s, start, end)) {
                continue;
            }

            path.push_back(s.substr(start, end - start + 1));

            dfs(s, end + 1, path, result);

            path.pop_back();
        }
    }

    vector<vector<string>> partition(string s) {
        vector<vector<string>> result;
        vector<string> path;
```

```
    dfs(s, 0, path, result);

    return result;
}
};
```

21.4 Complexity

Worst-case partitions are exponential.

Palindrome checking adds extra cost.

Time rough:

$O(n * 2^n)$

or more depending on substring cost.

Can optimize palindrome checks with DP.

22. Full Problem Walkthrough 11 — LeetCode 79: Word Search

22.1 Problem Summary

Given grid of characters and word, return true if word exists in grid.

Can move horizontally or vertically.

Cannot reuse same cell in one path.

22.2 How to Identify This Pattern

Need search path in grid.

Each step has up to 4 choices.

Need mark visited and unmark.

Use backtracking.

22.3 C++ Code

```
class Solution {
public:
    bool dfs(vector<vector<char>>& board, string& word,
             int r, int c, int index) {
        if (index == word.size()) {
            return true;
        }

        if (r < 0 || c < 0 || r >= board.size() || c >= board[0].size()) {
            return false;
        }

        if (board[r][c] != word[index]) {
```

```

        return false;
    }

    char saved = board[r][c];
    board[r][c] = '#';

    bool found =
        dfs(board, word, r + 1, c, index + 1) ||
        dfs(board, word, r - 1, c, index + 1) ||
        dfs(board, word, r, c + 1, index + 1) ||
        dfs(board, word, r, c - 1, index + 1);

    board[r][c] = saved;

    return found;
}

bool exist(vector<vector<char>>& board, string word) {
    int rows = board.size();
    int cols = board[0].size();

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (dfs(board, word, r, c, 0)) {
                return true;
            }
        }
    }

    return false;
}
};

```

22.4 Complexity

Let board size be $R * C$ and word length be L .

Time:

$O(R * C * 4^L)$

More tightly, after first move, branching is often 3.

Space:

$O(L)$

recursion depth.

22.5 Common Mistakes

- Not marking visited
 - Not restoring cell
 - Reusing same cell
 - Checking base case too late
 - Passing word by value unnecessarily
-

23. Full Problem Walkthrough 12 — LeetCode 51: N-Queens

23.1 Problem Summary

Place n queens on $n \times n$ board so no two queens attack each other.
Return all valid boards.

23.2 How to Identify This Pattern

Need place items under constraints.
At each row, choose a column.
If safe, place queen and recurse to next row.
Backtrack afterward.

23.3 Conflict Rules

Queens attack:

- same column
- same main diagonal
- same anti-diagonal

For cell (row, col):

main diagonal: $\text{row} - \text{col}$ anti diagonal: $\text{row} + \text{col}$
--

23.4 C++ Code

<pre>class Solution { public: void dfs(int n, int row, vector<string>& board, vector<vector<string>>& result, unordered_set<int>& cols, unordered_set<int>& diag1, unordered_set<int>& diag2) { if (row == n) { result.push_back(board); return; } } };</pre>
--

```

    }

    for (int col = 0; col < n; col++) {
        if (cols.count(col) ||
            diag1.count(row - col) ||
            diag2.count(row + col)) {
            continue;
        }

        board[row][col] = 'Q';
        cols.insert(col);
        diag1.insert(row - col);
        diag2.insert(row + col);

        dfs(n, row + 1, board, result, cols, diag1, diag2);

        board[row][col] = '.';
        cols.erase(col);
        diag1.erase(row - col);
        diag2.erase(row + col);
    }
}

vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> result;
    vector<string> board(n, string(n, '.'));

    unordered_set<int> cols;
    unordered_set<int> diag1;
    unordered_set<int> diag2;

    dfs(n, 0, board, result, cols, diag1, diag2);

    return result;
}
};

```

23.5 Complexity

Backtracking complexity is roughly:

$O(n!)$

Space:

$O(n)$

excluding output and board.

23.6 Common Mistakes

- Checking whole board every time
- Wrong diagonal formulas
- Forgetting to remove from sets
- Placing multiple queens in same row

24. Full Problem Walkthrough 13 — LeetCode 37: Sudoku Solver

24.1 Problem Summary

Fill a 9x9 Sudoku board.

Rules:

- each row contains digits 1-9 once
- each column contains digits 1-9 once
- each 3x3 box contains digits 1-9 once

24.2 How to Identify This Pattern

Need fill cells under constraints.

Try possible digit.

If valid, recurse.

If stuck, undo.

This is backtracking.

24.3 C++ Code

```
class Solution {
public:
    bool isValid(vector<vector<char>>& board, int row, int col, char digit) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == digit) {
                return false;
            }

            if (board[i][col] == digit) {
                return false;
            }

            int boxRow = 3 * (row / 3) + i / 3;
            int boxCol = 3 * (col / 3) + i % 3;
```

```

        if (board[boxRow][boxCol] == digit) {
            return false;
        }
    }

    return true;
}

bool solve(vector<vector<char>>& board) {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] != '.') {
                continue;
            }

            for (char digit = '1'; digit <= '9'; digit++) {
                if (isValid(board, row, col, digit)) {
                    board[row][col] = digit;

                    if (solve(board)) {
                        return true;
                    }

                    board[row][col] = '.';
                }
            }

            return false;
        }
    }

    return true;
}

void solveSudoku(vector<vector<char>>& board) {
    solve(board);
}
};

```

24.4 Complexity

Worst-case is exponential.
But constraints are fixed 9x9.

Backtracking with pruning is accepted.

24.5 Common Mistakes

- Not returning false after trying all digits
 - Not undoing digit
 - Wrong 3x3 box indexing
 - Continuing after solution found
-

25. Full Problem Walkthrough 14 — LeetCode 93: Restore IP Addresses

25.1 Problem Summary

Given string of digits, return all valid IP addresses.

An IP address has 4 parts.

Each part:

- between 0 and 255
- no leading zero unless part is exactly "0"

Example:

```
s = "25525511135"
```

Output:

```
["255.255.11.135","255.255.111.35"]
```

25.2 How to Identify This Pattern

Need partition string into 4 valid parts.

At each step choose segment length 1, 2, or 3.

Use backtracking.

25.3 C++ Code

```
class Solution {
public:
    bool isValid(string part) {
        if (part.size() > 1 && part[0] == '0') {
            return false;
        }

        int value = stoi(part);

        return value <= 255;
    }

    void dfs(string& s, int index,
            vector<string>& path,
```

```

        vector<string>& result) {
    if (path.size() == 4) {
        if (index == s.size()) {
            string ip = path[0] + "." + path[1] + "." + path[2] + "." + path[3];
            result.push_back(ip);
        }

        return;
    }

    for (int len = 1; len <= 3; len++) {
        if (index + len > s.size()) {
            break;
        }

        string part = s.substr(index, len);

        if (!isValid(part)) {
            continue;
        }

        path.push_back(part);

        dfs(s, index + len, path, result);

        path.pop_back();
    }
}

vector<string> restoreIpAddresses(string s) {
    vector<string> result;
    vector<string> path;

    dfs(s, 0, path, result);

    return result;
}
};

```

25.4 Complexity

There are at most 3 choices for each of 4 parts.

Time is effectively constant upper bound:

O(1)

relative to fixed IP size, though string processing exists.

25.5 Common Mistakes

- Allowing leading zeroes
- Allowing value greater than 255
- Not requiring exactly 4 parts
- Not consuming whole string

26. Full Problem Walkthrough 15 — LeetCode 526: Beautiful Arrangement

26.1 Problem Summary

For numbers 1 to n, count arrangements where for each position i:

`perm[i] % i == 0`

or:

`i % perm[i] == 0`

Return count.

26.2 How to Identify This Pattern

Need count valid permutations under constraints.

Use backtracking with used numbers.

26.3 C++ Code

```
class Solution {
public:
    void dfs(int n, int position, vector<bool>& used, int& count) {
        if (position > n) {
            count++;
            return;
        }

        for (int num = 1; num <= n; num++) {
            if (used[num]) {
                continue;
            }

            if (num % position != 0 && position % num != 0) {
```

```

        continue;
    }

    used[num] = true;

    dfs(n, position + 1, used, count);

    used[num] = false;
}
}

int countArrangement(int n) {
    vector<bool> used(n + 1, false);
    int count = 0;

    dfs(n, 1, used, count);

    return count;
}
};

```

26.4 Complexity

Worst-case:

$O(n!)$

Pruning reduces actual work.

27. Full Problem Walkthrough 16 — LeetCode 473: Matchsticks to Square

27.1 Problem Summary

Given matchsticks, return true if they can form a square.
Each side must have equal length.
Each matchstick must be used once.

27.2 How to Identify This Pattern

Need assign sticks into 4 groups with equal sum.
This is backtracking over buckets.

27.3 C++ Code

```
class Solution {
public:
    bool dfs(vector<int>& matchsticks, int index,
             vector<int>& sides, int target) {
        if (index == matchsticks.size()) {
            return sides[0] == target &&
                sides[1] == target &&
                sides[2] == target &&
                sides[3] == target;
        }

        int stick = matchsticks[index];

        for (int i = 0; i < 4; i++) {
            if (sides[i] + stick > target) {
                continue;
            }

            sides[i] += stick;

            if (dfs(matchsticks, index + 1, sides, target)) {
                return true;
            }

            sides[i] -= stick;

            if (sides[i] == 0) {
                break;
            }
        }

        return false;
    }

    bool makesquare(vector<int>& matchsticks) {
        int sum = accumulate(matchsticks.begin(), matchsticks.end(), 0);

        if (sum % 4 != 0) {
            return false;
        }

        int target = sum / 4;
```

```
sort(matchsticks.rbegin(), matchsticks.rend());

vector<int> sides(4, 0);

return dfs(matchsticks, 0, sides, target);
}
};
```

27.4 Why Sort Descending?

Place larger sticks first.

This causes invalid branches to fail earlier.

27.5 Why Break if Side Returns to 0?

If trying an empty side failed, trying another empty side is symmetric and unnecessary.

28. Full Problem Walkthrough 17 — LeetCode 698: Partition to K Equal Sum Subsets

28.1 Problem Summary

Return true if array can be partitioned into k non-empty subsets with equal sum.

28.2 How to Identify This Pattern

Need assign numbers to k buckets with equal target.

Similar to Matchsticks to Square.

28.3 C++ Code

```
class Solution {
public:
    bool dfs(vector<int>& nums, int index,
            vector<int>& buckets, int target) {
        if (index == nums.size()) {
            return true;
        }

        int x = nums[index];

        for (int i = 0; i < buckets.size(); i++) {
```

```

        if (buckets[i] + x > target) {
            continue;
        }

        buckets[i] += x;

        if (dfs(nums, index + 1, buckets, target)) {
            return true;
        }

        buckets[i] -= x;

        if (buckets[i] == 0) {
            break;
        }
    }

    return false;
}

bool canPartitionKSubsets(vector<int>& nums, int k) {
    int sum = accumulate(nums.begin(), nums.end(), 0);

    if (sum % k != 0) {
        return false;
    }

    int target = sum / k;

    sort(nums.rbegin(), nums.rend());

    if (nums[0] > target) {
        return false;
    }

    vector<int> buckets(k, 0);

    return dfs(nums, 0, buckets, target);
}
};

```

28.4 Complexity

Worst-case exponential.
Pruning is essential.

29. Full Problem Walkthrough 18 — LeetCode 282: Expression Add Operators

29.1 Problem Summary

Given numeric string num and target.

Add operators +, -, * between digits so expression evaluates to target.

Return all valid expressions.

Example:

```
num = "123"  
target = 6
```

Output:

```
["1+2+3", "1*2*3"]
```

29.2 How to Identify Backtracking

Need generate all expressions.

At each split, choose:

- next number
- operator before it

Need handle multiplication precedence.

29.3 Key State

Track:

- current expression string
- current evaluated value
- previous operand for multiplication correction

For multiplication:

```
1 + 2 * 3
```

When adding *3, previous +2 must be replaced by +6.

Formula:

```
newValue = value - prev + prev * current
```

29.4 C++ Code

```
class Solution {  
public:  
    void dfs(string& num, int target, int index,  
            long long value,
```

```

        long long prev,
        string path,
        vector<string>& result) {
    if (index == num.size()) {
        if (value == target) {
            result.push_back(path);
        }

        return;
    }

    for (int end = index; end < num.size(); end++) {
        if (end > index && num[index] == '0') {
            break;
        }

        string part = num.substr(index, end - index + 1);
        long long current = stoll(part);

        if (index == 0) {
            dfs(num, target, end + 1, current, current, part, result);
        } else {
            dfs(num, target, end + 1,
                value + current,
                current,
                path + "+" + part,
                result);

            dfs(num, target, end + 1,
                value - current,
                -current,
                path + "-" + part,
                result);

            dfs(num, target, end + 1,
                value - prev + prev * current,
                prev * current,
                path + "*" + part,
                result);
        }
    }
}

```

```

vector<string> addOperators(string num, int target) {
    vector<string> result;

    dfs(num, target, 0, 0, 0, "", result);

    return result;
}
};

```

29.5 Complexity

Exponential.

At each gap, multiple choices exist.

Use long long to avoid overflow.

29.6 Common Mistakes

- Ignoring multiplication precedence
- Allowing numbers with leading zeroes
- Using int instead of long long
- Adding operator before first number

30. Mini Problem 1 — Combination Sum III

Choose k numbers from 1 to 9 that sum to n.

```

class Solution {
public:
    void dfs(int start, int k, int target,
            vector<int>& path,
            vector<vector<int>>& result) {
        if (path.size() == k) {
            if (target == 0) {
                result.push_back(path);
            }

            return;
        }

        for (int i = start; i <= 9; i++) {
            if (i > target) {
                break;
            }

```

```

        path.push_back(i);

        dfs(i + 1, k, target - i, path, result);

        path.pop_back();
    }
}

vector<vector<int>> combinationSum3(int k, int n) {
    vector<vector<int>> result;
    vector<int> path;

    dfs(1, k, n, path, result);

    return result;
}
};

```

31. Mini Problem 2 — Factor Combinations

At each step, choose a factor starting from previous factor to avoid duplicate orders.

```

void dfs(int n, int start, vector<int>& path, vector<vector<int>>& result) {
    if (n == 1) {
        if (path.size() > 1) {
            result.push_back(path);
        }

        return;
    }

    for (int factor = start; factor <= n; factor++) {
        if (n % factor != 0) {
            continue;
        }

        path.push_back(factor);
        dfs(n / factor, factor, path, result);
        path.pop_back();
    }
}

```

```
}  
}
```

32. Mini Problem 3 — Backtracking With Boolean Return

Used when only need existence.

```
bool dfs(state) {  
    if (goal) {  
        return true;  
    }  
  
    for (choice : choices) {  
        make choice;  
  
        if (dfs(nextState)) {  
            return true;  
        }  
  
        undo choice;  
    }  
  
    return false;  
}
```

Examples:

- Word Search
- Sudoku Solver
- Matchsticks to Square

33. Common Edge Cases for Backtracking

33.1 Empty Input

Subsets of empty list:

```
[]
```

Letter combinations for empty digits:

```
[]
```

33.2 Duplicates

Sort and skip duplicates at same recursion level.

33.3 Reuse Allowed vs Not Allowed

Combination Sum I:

```
dfs(i, ...)
```

Combination Sum II:

```
dfs(i + 1, ...)
```

33.4 Leading Zeroes

Important for:

- Restore IP Addresses
- Expression Add Operators

33.5 Target Becomes Negative

Prune if candidates are positive.

33.6 Need to Restore State

Every mutation must be undone.

Examples:

```
path.pop_back();  
used[i] = false;  
board[r][c] = saved;
```

34. Common Mistakes and Wrong Code

Mistake 1 — Forgetting Unchoose Step

Wrong:

```
path.push_back(x);  
dfs(...);
```

Correct:

```
path.push_back(x);  
dfs(...);  
path.pop_back();
```

Mistake 2 — Duplicate Skip at Wrong Level

Correct for subsets/combinations:

```
if (i > start && nums[i] == nums[i - 1]) {  
    continue;  
}
```

Mistake 3 — Using i + 1 When Reuse Is Allowed

Combination Sum I needs:

```
dfs(i, ...)
```

Mistake 4 — Not Marking Visited in Grid

Word Search must mark cell before exploring neighbors.

Mistake 5 — Not Restoring Grid

Wrong:

```
board[r][c] = '#';  
return found;
```

Correct:

```
board[r][c] = saved;  
return found;
```

Mistake 6 — Building Invalid Then Filtering

Better to generate only valid states using constraints.

Example:

Generate Parentheses should not generate all strings of parentheses and filter.

35. Interview Explanation Templates

35.1 General Backtracking

I build a partial solution recursively. At each step, I try each valid choice, recurse, and then undo the choice so the next branch starts from a clean state.

35.2 Subsets

For each element, I decide whether to include it or skip it. The recursion explores all combinations of those choices.

35.3 Permutations

I build the permutation one position at a time. At each position, I choose one unused number, mark it used, recurse, then unmark it.

35.4 Combination Sum

I recursively choose candidates while reducing the remaining target. If reuse is allowed, I recurse with the same index. If not, I recurse with the next index.

35.5 Word Search

I run DFS from each cell. At each step, the current cell must match the current character. I temporarily mark the cell visited, explore four directions, then restore it.

35.6 N-Queens

I place one queen per row. I use sets to track used columns and diagonals, so I can check whether a placement is safe in $O(1)$.

36. Complexity Patterns

Subsets	$O(n * 2^n)$	$O(n)$
Subsets II	$O(n * 2^n)$	$O(n)$
Permutations	$O(n * n!)$	$O(n)$
Permutations II	$O(n * \text{unique permutations})$	$O(n)$
Combinations	$O(k * C(n, k))$	$O(k)$
Combination Sum	exponential	$O(\text{target} / \text{minCandidate})$
Letter Combinations	$O(4^n * n)$	$O(n)$
Generate Parentheses	$O(\text{Catalan}(n) * n)$	$O(n)$
Palindrome Partitioning	$O(n * 2^n)$ roughly	$O(n)$
Word Search	$O(R * C * 4^L)$	$O(L)$
N-Queens	$O(n!)$	$O(n)$
Sudoku Solver	exponential fixed board	$O(1)$ board size
Partition K Subsets	exponential	$O(k + n)$

37. Time-Boxed Practice Guidance

Intro Backtracking

Target:

20–35 minutes

Problems:

- Subsets
- Combinations
- Permutations
- Letter Combinations

Core Medium Backtracking

Target:

35–55 minutes

Problems:

- Combination Sum
- Combination Sum II
- Generate Parentheses
- Palindrome Partitioning
- Word Search

Hard Backtracking

Target:

60–90 minutes

Problems:

- N-Queens
- Sudoku Solver
- Matchsticks to Square
- Partition to K Equal Sum Subsets
- Expression Add Operators

38. Expanded Practice Set

38.1 Must-Do Medium

32. Subsets
33. Subsets II
34. Permutations
35. Permutations II
36. Combinations
37. Combination Sum
38. Combination Sum II
39. Letter Combinations of a Phone Number
40. Generate Parentheses
41. Palindrome Partitioning
42. Word Search
43. Restore IP Addresses

38.2 Strong Interview Practice

44. N-Queens
45. Sudoku Solver
46. Beautiful Arrangement
47. Matchsticks to Square
48. Partition to K Equal Sum Subsets
49. Combination Sum III
50. Word Search II, trie + backtracking
51. Gray Code

38.3 Hard / Stretch

52. Expression Add Operators
53. Remove Invalid Parentheses
54. N-Queens II
55. Sudoku Solver
56. Word Search II
57. Split Array into Fibonacci Sequence
58. Verbal Arithmetic Puzzle

39. Suggested Study Order

Step 1 — Simple Generation

Start with:

- 60. Subsets
- 61. Combinations
- 62. Permutations

Goal:

Understand path, choices, and used array.

Step 2 — Duplicate Handling

Solve:

- 63. Subsets II
- 64. Permutations II
- 65. Combination Sum II

Goal:

Learn sorting and duplicate skipping.

Step 3 — Target-Based Search

Solve:

- 66. Combination Sum
- 67. Combination Sum III
- 68. Restore IP Addresses

Goal:

Learn pruning with remaining target.

Step 4 — String Construction

Solve:

- 69. Letter Combinations
- 70. Generate Parentheses
- 71. Palindrome Partitioning

Goal:

Build valid strings and partitions.

Step 5 — Grid / Board Search

Solve:

- 72. Word Search
- 73. N-Queens
- 74. Sudoku Solver

Goal:

Learn visited marking and constraint sets.

Step 6 — Hard Constraint Assignment

Solve:

- 75. Matchsticks to Square

76. Partition to K Equal Sum Subsets

77. Expression Add Operators

Goal:

Practice pruning and advanced state.

40. Pattern Decision Table

all subsets	include/exclude or for-loop	Subsets
duplicates in subsets	sort + skip same level	Subsets II
all orderings	used array	Permutations
duplicate permutations	sort + used skip	Permutations II
choose k numbers	start index	Combinations
sum target, reuse allowed	recurse with i	Combination Sum
sum target, no reuse	recurse with i + 1	Combination Sum II
phone digits	choices per digit	Letter Combinations
valid parentheses	constrained string build	Generate Parentheses
split string	choose cut	Palindrome Partitioning
grid path	DFS + visited	Word Search
board placement	row-by-row constraints	N-Queens
fill board	try digits + undo	Sudoku
assign to buckets	bucket backtracking	Partition K Subsets

41. Final Cheat Sheet

[Choose](#) / [Explore](#) / [Unchoose](#)

```
path.push_back(choice);
```

```
dfs(nextState);
```

```
path.pop_back();
```

Used Array

```
if (used[i]) {  
    continue;  
}  
  
used[i] = true;  
path.push_back(nums[i]);  
  
dfs(...);  
  
path.pop_back();  
used[i] = false;
```

Skip Duplicates in Same Level

```
if (i > start && nums[i] == nums[i - 1]) {  
    continue;  
}
```

Reuse Allowed

```
dfs(i, target - candidates[i]);
```

Reuse Not Allowed

```
dfs(i + 1, target - candidates[i]);
```

Grid Visit

```
char saved = board[r][c];  
board[r][c] = '#';  
  
dfs(...);  
  
board[r][c] = saved;
```

N-Queens Diagonals

```
diag1 = row - col;  
diag2 = row + col;
```

42. What to Master Before Chapter 20

Before moving to Tree DFS, make sure you can:

- Explain choose / explore / unchoose
- Use path vector correctly

- Push and pop state
 - Use used array for permutations
 - Sort and skip duplicates
 - Know when reuse is allowed
 - Know when to use i vs $i + 1$
 - Solve Subsets
 - Solve Permutations
 - Solve Combination Sum
 - Solve Generate Parentheses
 - Solve Word Search
 - Solve Palindrome Partitioning
 - Understand pruning
 - Explain exponential complexity
 - Restore modified state after recursion
-

43. Core Takeaway

Backtracking is recursive trial and error with cleanup.

The core pattern is:

choose
explore
unchoose

Use it when a problem asks for all valid possibilities or requires searching through choices under constraints.

Backtracking is powerful because it systematically explores the decision tree.

It is expensive because the decision tree is often exponential.

The skill is not only writing recursion.

The skill is knowing:

- what the choices are
- what state to carry
- when a solution is complete
- when to prune
- how to undo state correctly

Backtracking becomes much easier when you can clearly define:

current path
remaining choices
validity constraints
base case